Cloud Data Engineer

Assignment

Name:ANUPKUMAR SHETTY

USN:23BTRIS005 DEPT: IMFORMATION SCIENCE

HCL N.O: HCLTFP2298086

SECTION A: ADVANCED SQL || Maximum Marks: 20

Question 1: Database Design and Advanced Queries (8 Marks) Create a database named EnterpriseDB. Create the following tables.

Employees

Column	Description
EmployeeID	Unique employee identifier
EmployeeName	Employee name
DepartmentID	Department identifier
Salary	Monthly salary
JoiningDate	Date of joining
ManagerID	Employee ID of the manager

Departments

Column	Description
DepartmentID	Unique department identifier
DepartmentName	Department name
Location	Department location

Requirements: • Define appropriate data types and constraints.

• Set primary keys and an appropriate foreign key.

• Insert at least 15 employee records and five department records.

• Include employees with different salaries, departments, and managers.

• Include at least one department without employees.

Creating database and table and also inserting the values:

```sql
CREATE DATABASE EnterpriseDB;
USE EnterpriseDB;

CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL,
    Location VARCHAR(50) NOT NULL
);

CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(100) NOT NULL,
    DepartmentID INT,
    Salary DECIMAL(10,2) NOT NULL,
    JoiningDate DATE NOT NULL,
    ManagerID INT,
    FOREIGN KEY (DepartmentID)
        REFERENCES Departments(DepartmentID),
    FOREIGN KEY (ManagerID)
        REFERENCES Employees(EmployeeID)
);
```

```sql
INSERT INTO Departments VALUES
(1, 'IT', 'Bengaluru'),
(2, 'HR', 'Mumbai'),
(3, 'Finance', 'Chennai'),
(4, 'Marketing', 'Hyderabad'),
(5, 'Research', 'Pune');

INSERT INTO Employees VALUES
(101, 'Anup', 1, 90000, '2022-01-15', NULL),
(105, 'Dhanush', 2, 75000, '2021-03-10', NULL),
(108, 'Druvanth', 3, 85000, '2020-07-20', NULL),
(112, 'Ashish', 4, 70000, '2023-02-01', NULL),
(102, 'Varun', 1, 60000, '2023-05-12', 101),
(103, 'Naveen', 1, 90000, '2022-08-18', 101),
(104, 'Vaishnavi', 1, 75000, '2024-01-10', 101),
(106, 'Shristy', 2, 45000, '2022-04-25', 105),
(107, 'Anup', 2, 55000, '2023-09-14', 105),
(109, 'Dhanush', 3, 50000, '2021-11-30', 108),
(110, 'Druvanth', 3, 65000, '2024-06-05', 108),
(111, 'Ashish', 3, 85000, '2022-12-11', 108),
(113, 'Varun', 4, 40000, '2023-07-19', 112),
(114, 'Naveen', 4, 70000, '2021-10-08', 112),
(115, 'Shristy', 4, 50000, '2024-03-22', 112);
```

Tasks:a.Display employee names, department names, and manager names using joins. (2 mar

```sql
SELECT
    e.EmployeeName,
    d.DepartmentName,
    m.EmployeeName AS ManagerName
FROM Employees e
JOIN Departments d
ON e.DepartmentID = d.DepartmentID
LEFT JOIN Employees m
ON e.ManagerID = m.EmployeeID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeName	DepartmentName	ManagerName
Anup	IT	NULL
Varun	IT	Anup
Naveen	IT	Anup
Vaishnavi	IT	Anup
Dhanush	HR	NULL
Shristy	HR	Dhanush
Anup	HR	Dhanush
Druvanth	Finance	NULL

b. Find employees earning more than their department's average salary. (2 marks)

```
56 • SELECT EmployeeName, DepartmentID, Salary
57   FROM Employees
58   WHERE Salary > (
59       SELECT AVG(Salary)
60       FROM Employees AS e2
61       WHERE e2.DepartmentID = Employees.DepartmentID
62   )
63   ORDER BY DepartmentID, Salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeName	DepartmentID	Salary
Anup	1	90000.00
Naveen	1	90000.00
Dhanush	2	75000.00
Druvanth	3	85000.00
Ashish	3	85000.00
Ashish	4	70000.00
Naveen	4	70000.00

c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. (2 marks)

```
65 • SELECT MAX(Salary) AS SecondHighestSalary
66   FROM Employees
67   WHERE Salary < (
68       SELECT MAX(Salary)
69       FROM Employees
70   );
71
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

SecondHighestSalary
85000.00

d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff. (2 marks)

```
72 •  SELECT *
73    FROM (
74        SELECT *,
75        DENSE_RANK() OVER (
76            PARTITION BY DepartmentID
77            ORDER BY Salary DESC
78        ) AS SalaryRank
79        FROM Employees
80    ) AS T
81    WHERE SalaryRank <= 3;
```

sult Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeID	EmployeeName	DepartmentID	Salary	JoiningDate	ManagerID	SalaryRan
101	Anup	1	1)0000.00	2022-01-15	NULL	1
103	Naveen	1	90000.00	2022-08-18	101	1
104	Vaishnavi	1	75000.00	2024-01-10	101	2
102	Varun	1	60000.00	2023-05-12	101	3
105	Dhanush	2	75000.00	2021-03-10	NULL	1
107	Anup	2	55000.00	2023-09-14	105	2
106	Shristy	2	45000.00	2022-04-25	105	3
108	Druvanth	3	85000.00	2020-07-20	NULL	1

Question 2:

Sales Analytics and Query Optimization (6 Marks)

Create a Sales table with the following columns:

- SaleID
- CustomerID
- SaleDate
- ProductName
- Quantity
- UnitPrice

Creating database and table and inserting it:

```
CREATE database SalesAnalytics;
USE SalesAnalytics;
CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    CustomerID INT,
    SaleDate DATE,
    ProductName VARCHAR(100),
    Quantity INT,
    UnitPrice DECIMAL(10,2)
);
INSERT INTO Sales
(SaleID, CustomerID, SaleDate, ProductName, Quantity, UnitPrice)
VALUES
(1, 101, '2026-01-08','keyboard', 2, 1000),
(2, 102, '2026-01-06', 'Mouse', 5, 500),
(3, 103, '2026-01-08', 'Keyboard', 3, 1200),
(4, 101, '2026-01-10', 'Monitor', 2, 12000),
(5, 104, '2026-01-12', 'Laptop', 1, 55000),
(6, 102, '2026-02-01', 'Keyboard', 2, 1200),
(7, 105, '2026-02-03', 'Mouse', 10, 500),
(8, 103, '2026-02-05', 'Laptop', 1, 55000),
(9, 101, '2026-02-10', 'Mouse', 4, 500),
(10, 104, '2026-02-15', 'Monitor', 1, 12000),
(11, 105, '2026-03-01', 'Keyboard', 5, 1200),
```

Task

a. Calculate total revenue for each product. (2 marks)

```
31 • SELECT
32       ProductName,
33       SUM(UnitPrice*Quantity) AS TotalRevenue
34   FROM Sales
35   GROUP BY ProductName;
36
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	TotalRevenue
keyboard	16400.00
Mouse	12500.00
Monitor	72000.00
Laptop	220000.00

C. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks) SELECT * FROM Employees

WHERE YEAR(JoiningDate) = 2025;
Assume JoiningDate may be indexed.

```
SELECT *
FROM Sales
WHERE JoiningDate >= '2025-01-01'
  AND JoiningDate < '2026-01-01';
```

Question 3:

SQL Troubleshooting (6 Marks)

A company maintains millions of employee and transaction records. Its reporting queries are running slowly.

a. Explain how an execution plan helps identify expensive operations in a query. (2 marks)

→AS the execution plan act as the blueprint for the execution of the query .where it divide the query into smaller part and also it will arrange the query accordingly and it also allocate the resource which is used for retrieving the data.

B. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. (2 marks)

→you should create a single composite index that combines both columns department and salar . Composite index is the database tool that shortcuts searches by sorting data using two or more columns at the same time .i think we can use this for query filtering.

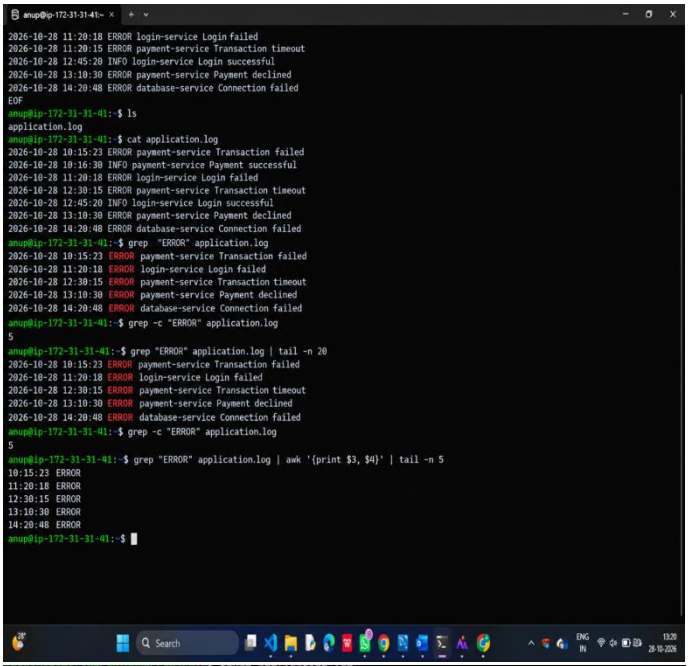
c.Write a query to identify duplicate customer email addresses and display only addresses occurring more than once.

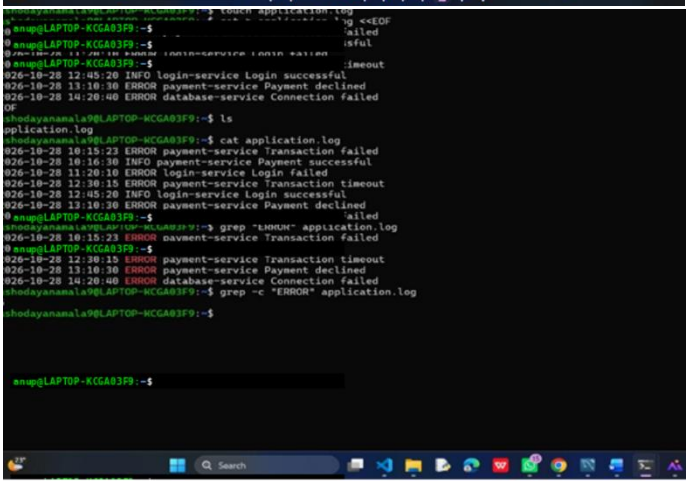
→To identify duplicate email addresses in a database, you need to group the records by the email column, count the occurrences of each email, and filter for counts greater than one.

SECTION B: LINUX || Maximum Marks: 15

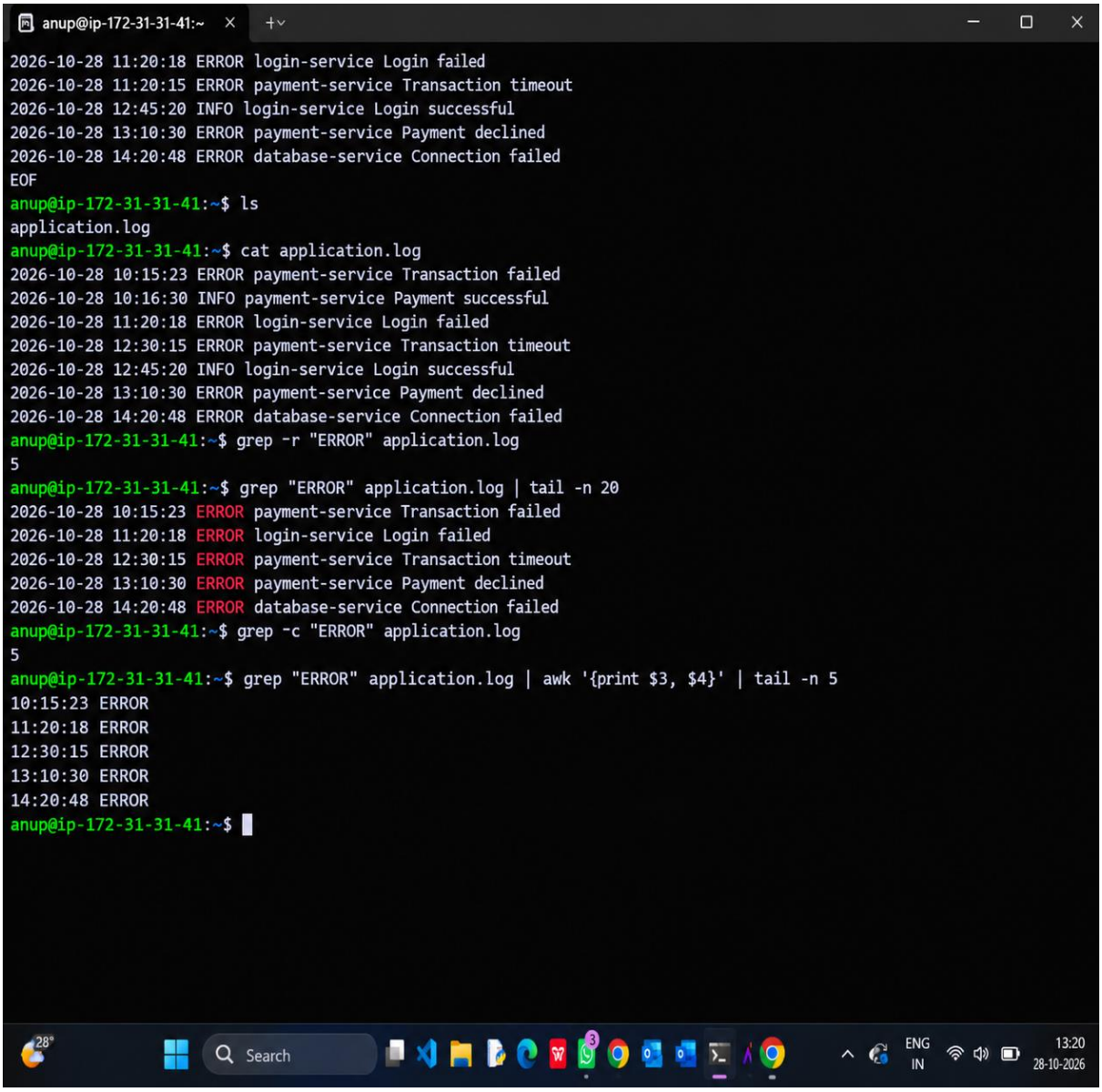
Question 4: Log Analysis (5 Marks)

a. Find all lines containing ERROR (1 mark)
   → grep "ERROR" /var/log/application.log
b. . Display the 20 most recent error entries in the file.
   → grep "ERROR" /var/log/application.log | tail -n 20
c. Count the total number of error entries.
   → grep -c "ERROR" /var/log/application.log
d. Extract error entries for a specified date into a separate file (1 mark)
   → grep "^2026-10-28 .*ERROR" /var/log/application.log
e. Explain how you would calculate the number of errors occurring in each hour. (1 mark)
   → grep "ERROR" /var/log/application.log | cut -d' ' -f1,2 | cut -d':' -f1 | sort | uniq -c


anup@ip-172-31-31-41:~
2026-10-28 11:20:18 ERROR login-service Login failed
2026-10-28 11:20:15 ERROR payment-service Transaction timeout
2026-10-28 12:45:20 INFO login-service Login successful
2026-10-28 13:10:30 ERROR payment-service Payment declined
2026-10-28 14:20:48 ERROR database-service Connection failed
EOF
anup@ip-172-31-31-41:~$ ls
application.log
anup@ip-172-31-31-41:~$ cat application.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:16:30 INFO payment-service Payment successful
2026-10-28 11:20:18 ERROR login-service Login failed
2026-10-28 12:30:15 ERROR payment-service Transaction timeout
2026-10-28 12:45:20 INFO login-service Login successful
2026-10-28 13:10:30 ERROR payment-service Payment declined
2026-10-28 14:20:48 ERROR database-service Connection failed
anup@ip-172-31-31-41:~$ grep "ERROR" application.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:20:18 ERROR login-service Login failed
2026-10-28 12:30:15 ERROR payment-service Transaction timeout
2026-10-28 13:10:30 ERROR payment-service Payment declined
2026-10-28 14:20:48 ERROR database-service Connection failed
anup@ip-172-31-31-41:~$ grep -c "ERROR" application.log
5
anup@ip-172-31-31-41:~$ grep "ERROR" application.log | tail -n 20
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:20:18 ERROR login-service Login failed
2026-10-28 12:30:15 ERROR payment-service Transaction timeout
2026-10-28 13:10:30 ERROR payment-service Payment declined
2026-10-28 14:20:48 ERROR database-service Connection failed
anup@ip-172-31-31-41:~$ grep -c "ERROR" application.log
5
anup@ip-172-31-31-41:~$ grep "ERROR" application.log | awk '{print $3, $4}' | tail -n 5
10:15:23 ERROR
11:20:18 ERROR
12:30:15 ERROR
13:10:30 ERROR
14:20:48 ERROR
anup@ip-172-31-31-41:~$
Search
ENG
IN
13:20
28-10-2026


anup@LAPTOP-KCGA03F9:~$
2026-10-28 12:45:20 INFO login-service Login successful
2026-10-28 13:10:30 ERROR payment-service Payment declined
2026-10-28 14:20:40 ERROR database-service Connection failed
EOF
shodayanamala9@LAPTOP-KCGA03F9:~$ ls
application.log
shodayanamala9@LAPTOP-KCGA03F9:~$ cat application.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:16:30 INFO payment-service Payment successful
2026-10-28 11:20:10 ERROR login-service Login failed
2026-10-28 12:30:15 ERROR payment-service Transaction timeout
2026-10-28 12:45:20 INFO login-service Login successful
2026-10-28 13:10:30 ERROR payment-service Payment declined
anup@LAPTOP-KCGA03F9:~$
2026-10-28 12:30:15 ERROR payment-service Transaction timeout
2026-10-28 13:10:30 ERROR payment-service Payment declined
2026-10-28 14:20:40 ERROR database-service Connection failed
shodayanamala9@LAPTOP-KCGA03F9:~$ grep -c "ERROR" application.log
shodayanamala9@LAPTOP-KCGA03F9:~$
anup@LAPTOP-KCGA03F9:~$
Search

```
anup@DESKTOP-T6LM3N9:~$ grep "ERROR" application.log | tail -n 20
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:20:10 ERROR login-service Login failed
2026-10-28 12:30:15 ERROR payment-service Transaction timeout
2026-10-28 13:10:30 ERROR payment-service Payment declined
2026-10-28 14:20:40 ERROR database-service Connection failed
anup@DESKTOP-T6LM3N9:~$
```





Question 5: Permissions and System Troubleshooting (5 Marks)

A Linux server hosts an application that processes confidential financial reports.

Perform the following tasks:

a. Create a group named finance and a report file, assuming you have the required privileges. **(1 mark)**

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. **(1 mark)**

c. Identify the processes consuming the most CPU. **(1 mark)**

d. Display memory usage and filesystem disk utilization. **(1 mark)**

e. Find files larger than 100 MB under a specified application directory. **(1 mark)**

Answer:

A. # 1. Create the group named 'finance'→ sudo groupadd finance
   # 2. Create an empty report file (e.g., financial_report.txt) →touch financial_report.txt
B. chmod 640 financial_report.txt
C. top
D. df -h
E. find /path/to/app/directory -type f -size +100M

SECTION C: PYSPARK DATA ENGINEERING || Maximum Marks: 20

Question 7: PySpark DataFrame Operations (6 Marks)

Create a PySpark DataFrame containing the following columns:

- EmployeeID
- EmployeeName
- Department
- Salary
- JoiningDate

Insert at least 15 sample records.

Write and execute a PySpark program to:

a. Display the schema and first five records. **(1 mark)**

b. Filter employees whose monthly salary is greater than ₹60,000. **(1 mark)**

c. Add an AnnualSalary column calculated from the monthly salary. **(1 mark)**

d. Group employees by department and calculate the average salary and employee count. **(2 marks)**

e. Display the five highest-paid employees. **(1 mark)**

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

# Create Spark session
spark = SparkSession.builder \
    .appName("Employee DataFrame Operations") \
    .master("local[*]") \
    .getOrCreate()

# Sample employee records
data = [
    (101, "Arun", "IT", 75000, "2023-01-15"),
    (102, "Priya", "HR", 45000, "2022-03-10"),
    (103, "Rahul", "IT", 85000, "2021-06-20"),
    (104, "Sneha", "Finance", 65000, "2023-07-01"),
    (105, "Kiran", "HR", 55000, "2022-11-12"),
    (106, "Divya", "IT", 95000, "2020-02-25"),
    (107, "Vijay", "Finance", 70000, "2021-09-18"),
    (108, "Anjali", "Marketing", 50000, "2023-04-05"),
    (109, "Ravi", "IT", 60000, "2022-08-14"),
    (110, "Meena", "Finance", 80000, "2020-12-11"),
    (111, "Suresh", "HR", 62000, "2021-05-22"),
    (112, "Pooja", "Marketing", 68000, "2022-02-16"),
    (113, "Amit", "IT", 72000, "2023-10-09"),
    (114, "Neha", "Finance", 58000, "2022-06-30"),
    (115, "Manoj", "Marketing", 90000, "2020-08-19")
]
```

```python
# Define DataFrame schema
columns = [
    "EmployeeID", "EmployeeName", "Department",
    "Salary", "JoiningDate"
]

df = spark.createDataFrame(data, columns)

# Convert JoiningDate from string to date
df = df.withColumn(
    "JoiningDate", col("JoiningDate").cast("date")
)

# (a) Display schema and first five records
print("=== a. SCHEMA ===")
df.printSchema()

print("=== FIRST FIVE RECORDS ===")
df.show(5, truncate=False)

# (b) Filter employees earning more than Rs. 60,000
print("=== b. SALARY GREATER THAN 60000 ===")
df.filter(col("Salary") > 60000).show(truncate=False)

# (c) Add AnnualSalary column
df_annual = df.withColumn(
    "AnnualSalary", col("Salary") * 12
)
```

```
# (c) Add AnnualSalary column
df_annual = df.withColumn(
    "AnnualSalary", col("Salary") * 12
)

print("=== c. ANNUAL SALARY ===")
df_annual.show(truncate=False)

# (d) Average salary and employee count by department
print("=== d. DEPARTMENT-WISE SUMMARY ===")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).orderBy("Department").show(truncate=False)

# (e) Display five highest-paid employees
print("=== e. TOP FIVE HIGHEST-PAID EMPLOYEES ===")
df.orderBy(col("Salary").desc()).show(5, truncate=False)

# Stop Spark session
spark.stop()
```

output

```
=== a. SCHEMA ===
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

=== FIRST FIVE RECORDS ===
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Arun        |IT        |75000 |2023-01-15 |
|102       |Priya       |HR        |45000 |2022-03-10 |
|103       |Rahul       |IT        |85000 |2021-06-20 |
|104       |Sneha       |Finance   |65000 |2023-07-01 |
|105       |Kiran       |HR        |55000 |2022-11-12 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
=== b. SALARY GREATER THAN 60000 ===
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Arun        |IT        |75000 |2023-01-15 |
|103       |Rahul       |IT        |85000 |2021-06-20 |
|104       |Sneha       |Finance   |65000 |2023-07-01 |
|106       |Divya       |IT        |95000 |2020-02-25 |
|107       |Vijay       |Finance   |70000 |2021-09-18 |
|110       |Meena       |Finance   |80000 |2020-12-11 |
|111       |Suresh      |HR        |62000 |2021-05-22 |
|112       |Pooja       |Marketing |68000 |2022-02-16 |
|113       |Amit        |IT        |72000 |2023-10-09 |
```

```
=== c. ANNUAL SALARY ===
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|101       |Arun        |IT        |75000 |2023-01-15 |900000      |
|102       |Priya       |HR        |45000 |2022-03-10 |540000      |
|103       |Rahul       |IT        |85000 |2021-06-20 |1020000     |
|104       |Sneha       |Finance   |65000 |2023-07-01 |780000      |
|105       |Kiran       |HR        |55000 |2022-11-12 |660000      |
|106       |Divya       |IT        |95000 |2020-02-25 |1140000     |
|107       |Vijay       |Finance   |70000 |2021-09-18 |840000      |
|108       |Anjali      |Marketing |50000 |2023-04-05 |600000      |
|109       |Ravi        |IT        |60000 |2022-08-14 |720000      |
|110       |Meena       |Finance   |80000 |2020-12-11 |960000      |
|111       |Suresh      |HR        |62000 |2021-05-22 |744000      |
|112       |Pooja       |Marketing |68000 |2022-02-16 |816000      |
|113       |Amit        |IT        |72000 |2023-10-09 |864000      |
|114       |Neha        |Finance   |58000 |2022-06-30 |696000      |
|115       |Manoj       |Marketing |90000 |2020-08-19 |1080000     |
+----------+------------+----------+------+-----------+------------+

=== d. DEPARTMENT-WISE SUMMARY ===
+----------+-----------------+-------------+
|Department|AverageSalary    |EmployeeCount|
+----------+-----------------+-------------+
|Finance   |68250.0          |4            |
|HR        |54000.0          |3            |
|IT        |77400.0          |5            |
|Marketing |69333.33333333333|3            |
+----------+-----------------+-------------+

=== e. TOP FIVE HIGHEST-PAID EMPLOYEES ===
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|106       |Divya       |IT        |95000 |2020-02-25 |
|115       |Manoj       |Marketing |90000 |2020-08-19 |
|103       |Rahul       |IT        |85000 |2021-06-20 |
|110       |Meena       |Finance   |80000 |2020-12-11 |
|101       |Arun        |IT        |75000 |2023-01-15 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

Question 8: Data Cleaning and Aggregation (7 Marks)

A company receives a CSV file named transactions.csv containing:

- TransactionID
- CustomerID
- TransactionDate
- Product
- Quantity
- UnitPrice

The file contains duplicate records, missing values, and incorrectly formatted dates.

Write a PySpark program to:

a. Read the CSV file using an appropriate schema and header configuration. **(1 mark)**

b. Identify and remove duplicate transactions using a suitable business key. **(1 mark)**

c. Handle missing values in required fields and explain your chosen approach. **(2 marks)**

d. Convert TransactionDate into a date type and handle invalid dates. **(1 mark)**

e. Calculate revenue using Quantity × UnitPrice. **(1 mark)**

f. Calculate total revenue by product and display the results in descending order. **(1 mark)**

Expected Output: Submit the program and show the cleaned data and aggregation results.

```python
# Install PySpark
!pip install pyspark -q

from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import (
    col, to_date, sum as _sum, expr
)

# Create Spark session
spark = SparkSession.builder \
    .appName("TransactionCleaning") \
    .getOrCreate()

# a. Define schema
schema = StructType([
    StructField("TransactionID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("TransactionDate", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True)
])

# Read CSV file
df = spark.read.csv(
    "/content/transactions.csv",
    header=True,
```

```python
# Read CSV file
df = spark.read.csv(
    "/content/transactions.csv",
    header=True,
    schema=schema,
    mode="PERMISSIVE"
)

print("Original Data:")
df.show(truncate=False)

# b. Remove duplicate transactions
df = df.dropDuplicates(["TransactionID"])

# c. Remove missing required fields
df = df.dropna(subset=[
    "TransactionID",
    "CustomerID",
    "TransactionDate",
    "Product",
    "Quantity",
    "UnitPrice"
])

# Remove invalid quantity and price
df = df.filter(
    (col("Quantity") > 0) &
    (col("UnitPrice") >= 0)
)
```

```python
# d. Convert date safely and remove invalid dates
df = df.withColumn(
    "TransactionDate",
    expr("try_to_timestamp(TransactionDate, 'yyyy-MM-dd')").cast("date")
)

df = df.dropna(subset=["TransactionDate"])

# e. Calculate revenue
df = df.withColumn(
    "Revenue",
    col("Quantity") * col("UnitPrice")
)

print("Cleaned Transactions:")
df.show(truncate=False)

# f. Calculate total revenue by product
result = df.groupBy("Product").agg(
    _sum("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc())

print("Total Revenue by Product:")
result.show(truncate=False)

# Stop Spark
spark.stop()
```

output

Original Data:

TransactionID	CustomerID	TransactionDate	Product	Quantity	UnitPrice
T001	C101	2026-10-01	Laptop	2	50000.0
T002	C102	2026-10-02	Mouse	5	500.0
T003	C103	2026-10-03	Laptop	1	50000.0
T004	C104	2026-10-04	Keyboard	3	1200.0
T005	C105	2026-10-05	Mouse	2	500.0
T006	C106	2026-10-06	Monitor	2	10000.0
T007	C107	2026-10-07	Keyboard	1	1200.0
T008	C108	2026-10-08	Laptop	1	50000.0
T009	C109	2026-10-09	Monitor	1	10000.0
T010	C110	2026-10-10	Mouse	4	500.0
T002	C102	2026-10-02	Mouse	5	500.0
T011	C111	2026-10-11	Keyboard	0	1200.0
T012	C112	2026-10-12	Monitor	2	-100.0
T013	C113	invalid-date	Mouse	1	500.0
T014	NULL	2026-10-14	Laptop	1	50000.0

Cleaned Transactions:

TransactionID	CustomerID	TransactionDate	Product	Quantity	UnitPrice	Revenue
T001	C101	2026-10-01	Laptop	2	50000.0	100000.0
T002	C102	2026-10-02	Mouse	5	500.0	2500.0
T003	C103	2026-10-03	Laptop	1	50000.0	50000.0
T004	C104	2026-10-04	Keyboard	3	1200.0	3600.0
T005	C105	2026-10-05	Mouse	2	500.0	1000.0
T006	C106	2026-10-06	Monitor	2	10000.0	20000.0
T007	C107	2026-10-07	Keyboard	1	1200.0	1200.0
T008	C108	2026-10-08	Laptop	1	50000.0	50000.0
T009	C109	2026-10-09	Monitor	1	10000.0	10000.0
T010	C110	2026-10-10	Mouse	4	500.0	2000.0

Total Revenue by Product:

Product	TotalRevenue
Laptop	200000.0
Monitor	30000.0
Mouse	5500.0
Keyboard	4800.0

Question 9: PySpark Joins and Performance (7 Marks)

A company maintains two datasets.

Customers

- CustomerID
- CustomerName
- City

Orders

- OrderID
- CustomerID
- OrderAmount
- OrderDate

Write a PySpark program to:

a. Load both datasets into DataFrames. **(1 mark)**

b. Join the datasets to display orders with customer details. **(2 marks)**

c. Identify customers who have never placed an order. **(1 mark)**

d. Calculate the total order value by city. **(1 mark)**

e. Explain when a broadcast join is useful and what conditions make it appropriate. **(1 mark)**

f. Explain the difference between transformations and actions, giving one example of each. **(1 mark)**

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum, broadcast

# Create Spark session
spark = SparkSession.builder.appName("CustomerOrders").getOrCreate()

# a. Create customer dataset
customers_data = [
    (1, "Rahul", "Bangalore"),
    (2, "Priya", "Mysore"),
    (3, "Amit", "Chennai"),
    (4, "Sneha", "Bangalore")
]

customers = spark.createDataFrame(
    customers_data, ["CustomerID", "CustomerName", "City"]
)

# Create orders dataset
orders_data = [
    (101, 1, 2500.0, "2026-10-01"),
    (102, 2, 1500.0, "2026-10-02"),
    (103, 1, 3000.0, "2026-10-05")
]

orders = spark.createDataFrame(
    orders_data, ["OrderID", "CustomerID", "OrderAmount", "OrderDate"]
)
```

rere

```python
print("CUSTOMERS")
customers.show()

print("ORDERS")
orders.show()

# b. Join orders with customer details
print("ORDERS WITH CUSTOMER DETAILS")
customer_orders = customers.join(orders, "CustomerID", "inner")
customer_orders.show()

# c. Customers who never placed an order
print("CUSTOMERS WITHOUT ORDERS")
customers.join(orders, "CustomerID", "left_anti").show()

# d. Total order value by city
print("TOTAL ORDER VALUE BY CITY")
customer_orders.groupBy("City").agg(
    sum("OrderAmount").alias("TotalOrderValue")
).show()

# e. Broadcast join
print("BROADCAST JOIN")
customers.join(broadcast(orders), "CustomerID", "inner").show()

# f. Transformation and action
print("ORDERS ABOVE 2000")
filtered_orders = orders.filter(orders.OrderAmount > 2000)  # Transformation
filtered_orders.show()  # Action
```

results

```
CUSTOMERS
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         1|       Rahul|Bangalore|
|         2|       Priya|   Mysore|
|         3|        Amit|  Chennai|
|         4|       Sneha|Bangalore|
+----------+------------+---------+

ORDERS
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    101|         1|     2500.0|2026-10-01|
|    102|         2|     1500.0|2026-10-02|
|    103|         1|     3000.0|2026-10-05|
+-------+----------+-----------+----------+

ORDERS WITH CUSTOMER DETAILS
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName|     City|OrderID|OrderAmount| OrderDate|
+----------+------------+---------+-------+-----------+----------+
|         1|       Rahul|Bangalore|    101|     2500.0|2026-10-01|
|         1|       Rahul|Bangalore|    103|     3000.0|2026-10-05|
|         2|       Priya|   Mysore|    102|     1500.0|2026-10-02|
+----------+------------+---------+-------+-----------+----------+
```

```
ORDERS WITH CUSTOMER DETAILS
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName|     City|OrderID|OrderAmount| OrderDate|
+----------+------------+---------+-------+-----------+----------+
|         1|       Rahul|Bangalore|    101|     2500.0|2026-10-01|
|         1|       Rahul|Bangalore|    103|     3000.0|2026-10-05|
|         2|       Priya|   Mysore|    102|     1500.0|2026-10-02|
+----------+------------+---------+-------+-----------+----------+

CUSTOMERS WITHOUT ORDERS
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         3|        Amit|  Chennai|
|         4|       Sneha|Bangalore|
+----------+------------+---------+

TOTAL ORDER VALUE BY CITY
+---------+---------------+
|     City|TotalOrderValue|
+---------+---------------+
|Bangalore|         5500.0|
|   Mysore|         1500.0|
```

SECTION D: AWS CLOUD SERVICES || Maximum Marks: 15

Question 10: S3 and IAM Security (5 Marks)

A company stores confidential employee reports in Amazon S3. Only authorized HR employees should be able to read these reports.

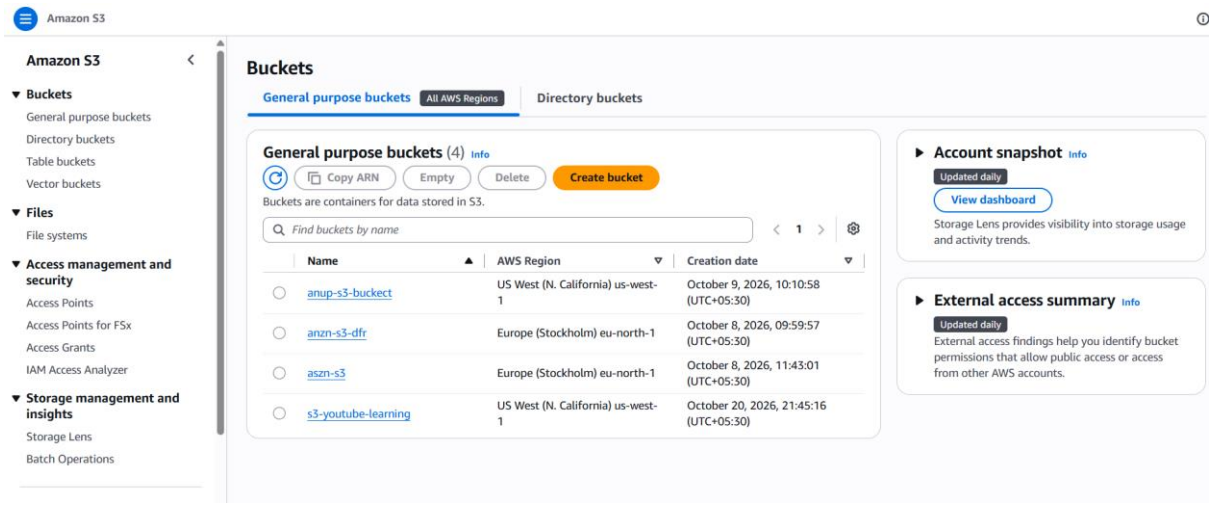
Perform or document the following:

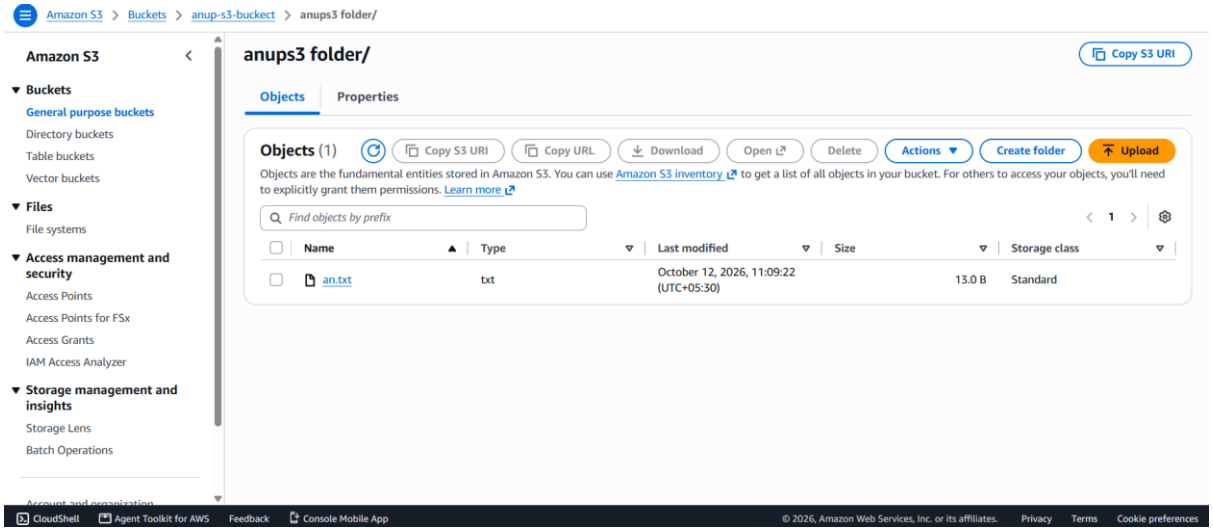
a. Create an S3 bucket with Block Public Access enabled and upload a sample file. **(1 mark)**

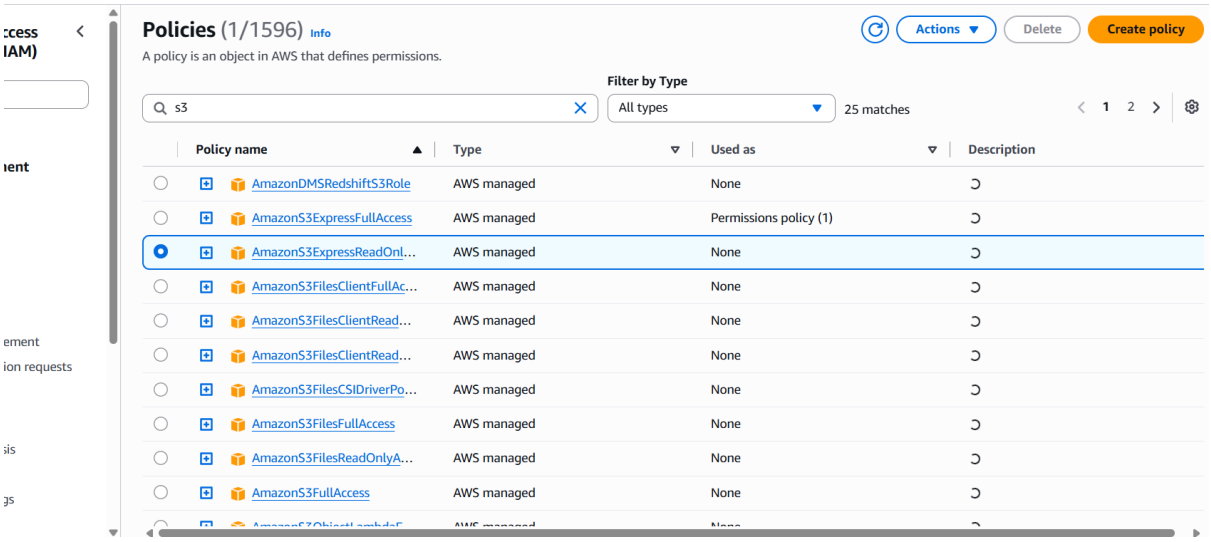
b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. **(2 marks)**

c. Explain how you would verify that an unauthorized user cannot read the file. **(1 mark)**

d. Explain how encryption at rest protects the stored data. **(1 mark)**







c.Explain how you would verify that an unauthorized user cannot read the file. (1 mark)

To verify that an unauthorized user cannot read the file, you can perform either of the following checks:

- **Attempt to open the file using an unauthorized account** and confirm that access is denied.

d. Explain how encryption at rest protects the stored data. (1 mark)

Encryption at rest protects stored data by converting it into an unreadable format (ciphertext) using a cryptographic algorithm.

- Prevents physical theft exposure: If a hard drive or storage medium is stolen, lost, or accessed outside the operating system, the data remains unreadable without the corresponding decryption key.
- Blocks unauthorized logical access: Even if an attacker bypasses network defenses or gains unauthorized access to the storage system, they cannot view the actual contents of the files without the correct key.

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks)

A web application hosted on EC2 becomes slow, and users report connectivity problems.

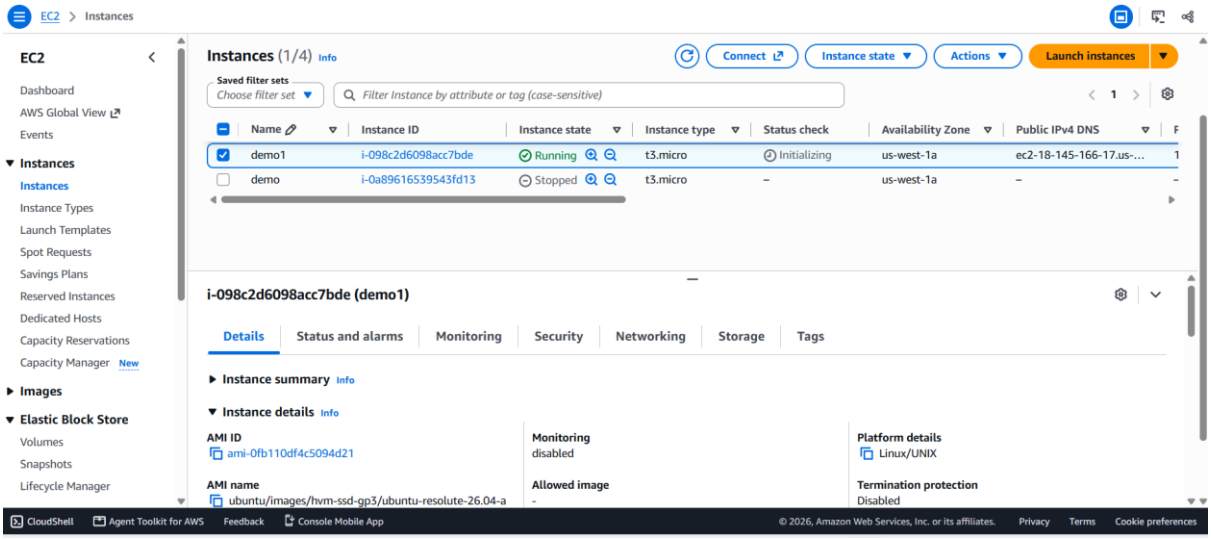
Perform or document the following:

a. Check the EC2 instance state and status checks. **(1 mark)**

b. Explain how security groups and route tables can affect application connectivity. **(1 mark)**

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period. **(2 marks)**

d. Identify one method for investigating the root cause of high CPU utilization. **(1 mark)**



b. Explain how security groups and route tables can affect application connectivity. (1 mark)

Security groups act as firewalls for your servers, while route tables act as road signs that direct network traffic. Together, they control whether your application can connect to the internet or other resources.

- Security Groups (The Guards):

They check who is allowed in or out. If a security group rule does not explicitly allow a specific type of traffic (like web traffic on port 80), the connection is blocked at the server's door.

- Route Tables (The Maps): They determine where the traffic goes. If a route table does not have a "road" (route) pointing toward a gateway or the internet, the network traffic will get lost and never reach its destination.

c.**Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period**.

o configure a CloudWatch alarm for high CPU utilization, follow these steps:

1. **Open the CloudWatch Console:** Navigate to the **Alarms** section in the AWS Management Console and click **Create alarm**.
2. **Select the Metric:** Click **Select metric**, choose **EC2**, select **Per-Instance Metrics**, and pick **CPUUtilization** for your specific instance.
3. **Set the Threshold:** Under the Conditions section, set the threshold type to **Static**, choose **Greater than (>),** and enter **85** as the value.
4. **Define the Evaluation Period:** Specify the **Period** (e.g., 5 minutes) and the **Evaluation Periods** (e.g., 2 out of 2) to determine how long the CPU must stay above 85% before triggering.
5. **Configure Actions:** Set up a notification (like sending an email via **Amazon SNS**) or an automated action (like an **Auto Scaling**

policy) to run when the alarm state is triggered.

6. **Name and Create:** Give the alarm a clear name and description, review the settings, and click **Create alarm**.

Question 12: Event-Driven Processing (5 Marks)

A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location.

Design a solution using AWS services.

a. Identify a suitable service for executing the processing code. **(1 mark)**

b. Explain how an S3 upload can trigger processing. **(1 mark)**

c. Explain the IAM permissions needed to read the input file and write the results. **(1 mark)**

d. Explain how errors and processing logs can be captured. **(1 mark)**

e. Explain how to prevent duplicate processing or recursive triggering. **(1 mark)**

a. Identify a suitable service for executing the processing code.

AWS Lambda is a suitable service for executing the processing code. It automatically runs Python or other code when triggered by an S3 file upload, without requiring server management.

b. Explain how an S3 upload can trigger processing. (1 mark)

Configure an S3 Event Notification to trigger an AWS Lambda function whenever a CSV file is uploaded to the S3 bucket. Lambda automatically executes the processing code.

c. Explain the IAM permissions needed to read the input file and write the results. (1 mark)

To read the input file and write the results, the IAM identity (user, group, or role) needs specific permissions targeting the source and destination storage resources.

Assuming the files are stored in Amazon S3, the minimum required IAM permissions are:

- Read (Input): s3:GetObject on the specific input bucket and file path to download or view the data.
- Write (Results): s3:PutObject on the destination bucket and path to save the output files.

d. Explain how errors and processing logs can be captured. (1 mark)

Use Amazon CloudWatch to capture Lambda execution logs and errors. It helps monitor processing failures and troubleshoot issues.

e. Explain how to prevent duplicate processing or recursive triggering. (1 mark)

To prevent duplicate processing or recursive triggering (especially in event-driven systems like AWS Lambda triggered by S3), use these strategies:

- Use separate buckets or folders (prefixes): Read your input files from a source location and write the output files to a completely different destination location so the output does not trigger the process again.
- Filter by file naming or extensions: Configure your event trigger to only respond to specific file types (e.g., .txt or .json) or specific naming patterns (e.g., input/ folder only).
- Track processed items (Idempotency): Save a record of the unique file name or transaction ID in a database (like DynamoDB). Before processing anything, check the database to ensure it has not been handled already.

SECTION E: AWS STEP FUNCTIONS || Maximum Marks: 15

Question 13: State Machine Design (7 Marks)

A company needs to process sales files through the following stages:

1. Validate the input file.
2. Transform the data.
3. Calculate total sales by product.
4. Store the results in Amazon S3.
5. Send a completion notification.

Design an AWS Step Functions state machine to coordinate the workflow.

a. Identify suitable states for validation, processing, storage, and notification. **(2 marks)**

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. **(1 mark)**

c. Explain how Retry and Catch can handle transient failures and permanent errors. **(2 marks)**

d. Explain how the state machine can invoke Lambda functions or other supported AWS services. **(1 mark)**

e. Explain how execution history helps troubleshoot a failed workflow. **(1 mark)**

Expected Output: Draw the workflow or submit an Amazon States Language (ASL) definition.

a. Suitable states (2 marks)

- Task states: Validate the file, transform data, calculate total sales, store results in S3, and send a notification.
- Choice state: Check whether the input file is valid.

b. Choice state (1 mark)

The Choice state checks the validation result. If valid, the workflow continues to processing; if invalid, it routes to the failure path.

c. Retry and Catch (2 marks)

- Retry: Automatically retries temporary failures, such as network errors.
- Catch: Handles permanent errors by routing the workflow to a failure state or sending an error notification.

d. Invoking AWS services (1 mark)

Step Functions uses Task states to invoke AWS Lambda functions or other supported AWS service integrations to execute each stage.

e. Execution history (1 mark)

Execution history records each state's input, output, status, and errors, helping developers identify and troubleshoot failed steps.

Question 14: Step Functions Implementation (8 Marks)

A data engineering team has developed three processing functions:

- ValidateFile
- TransformData

- LoadResults

The functions must execute in sequence.

Requirements:

- If validation fails, the workflow must stop.
- If transformation fails temporarily, the workflow must retry.
- If loading fails permanently, the workflow must record the failure.

Tasks:

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary. **(3 marks)**

b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts. **(2 marks)**

c. Configure a Catch transition for the loading task that routes to an error-handling state. **(1 mark)**

d. Explain how data from one task is passed to the next task. **(1 mark)**

e. Explain how you would test successful and failed executions. **(1 mark)**

Expected Output: Submit the JSON definition and explain the states, transitions, and error-handling strategy.

a. ASL JSON definition

```
{
 "Comment": "Sales processing workflow",
 "StartAt": "ValidateFile",
 "States": {
  "ValidateFile": {
   "Type": "Task",
   "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:ValidateFile",
   "Next": "TransformData",
   "Catch": [
    {
     "ErrorEquals": ["States.ALL"],
     "Next": "ValidationFailed"
    }
   ]
  },
  "TransformData": {
   "Type": "Task",
   "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:TransformData",
   "Retry": [
```

```
      {
        "ErrorEquals": ["Lambda.ServiceException",
                        "Lambda.TooManyRequestsException"],
        "IntervalSeconds": 2,
        "MaxAttempts": 3,
        "BackoffRate": 2
      }
    ],
    "Next": "LoadResults"
  },
  "LoadResults": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:REGION:ACCOUNT_ID:function:LoadResults",
    "Catch": [
      {
        "ErrorEquals": ["States.ALL"],
        "Next": "LoadingFailed"
      }
    ],
    "End": true
  },
  "ValidationFailed": {
    "Type": "Fail",
    "Error": "ValidationError",
    "Cause": "File validation failed"
  },
  "LoadingFailed": {
    "Type": "Fail",
    "Error": "LoadingError",
    "Cause": "Loading failed permanently"
  }
 }
}
```

b. Retry policy (2 marks)

The Retry field retries temporary transformation errors up to 3 times, starting after 2 seconds. The delay doubles after each retry because BackoffRate is 2.

c. Catch transition (1 mark)

The Catch field in LoadResults catches errors and redirects execution to LoadingFailed, where the workflow records the failure.

d. Passing data between tasks (1 mark)

Step Functions automatically passes a task's output as the next task's input by default. The ResultPath field can combine task results with the original input.

e. Testing executions (1 mark)

- Successful test: Provide a valid input and verify that all three tasks execute in sequence.
- Failure test: Simulate validation, transformation, and loading errors to verify stopping, retries, and failure handling.

SECTION F: MICROSOFT AZURE || Maximum Marks: 15

Question 15: Azure Data Lake Storage and Security (5 Marks)

a) Create an Azure Storage Account and enable Hierarchical Namespace for ADLS Gen2.

b)Create a filesystem or container and upload a sample CSV dataset.

c) Microsoft Entra ID identifies users, and Azure RBAC assigns permissions to access the data.

d)Test access using an unauthorized account and verify that access is denied

Question 16: Azure VM Troubleshooting (5 Marks)

a)Check the VM's provisioning state and power state in the Azure Portal.

b)Incorrect Network Security Group rules may block SSH or RDP connections.

c)Incorrect subnet settings or route tables can prevent the VM from connecting to the network.

d)Use Azure Bastion to connect securely without exposing the VM's public IP. Also check boot diagnostics and network settings.

Question 17: Azure Data Integration (5 Marks)

a)Use Azure Data Factory to automate and manage data movement.

b) Create a pipeline to copy the CSV file from its source into Azure Data Lake Storage Gen2.

c) Organize files into date-based folders, such as year/month/day.

d)Use mapping data flows to handle malformed records and replace or remove missing values.

e) Monitor pipeline runs in Azure Data Factory and validate the row count to ensure all records are copied.